

COMPTE RENDU DE TP

Chapitre 2 : Pipelines agiles, versionnement et orchestration avec dlt, dbt et Dagster

Informations
Module : MLOps & DataOps
Encadrant : Pr. Mohammed AIT DAOUD
Étudiante : SEKKAT ISRAA
Année universitaire : 2025-2026

Liste des figures et captures

- Capture 1** — pip freeze > requirements.txt
- Capture 2** — git init et configuration de la branche main
- Capture 3** — Premier commit Git — Initialisation du projet
- Capture 4** — Contenu du fichier ventes.csv
- Capture 5** — Exécution de python pipeline/ingest.py — Ingestion terminée
- Capture 6** — Commit Git — Ajout de l'étape d'ingestion
- Capture 7** — Exécution de python pipeline/validate.py — Validation réussie
- Capture 8** — Commit Git — Ajout de l'étape de validation minimale
- Capture 9** — Modèle ventes_clean.sql
- Capture 10** — Modèle ventes_resume.sql
- Capture 11** — dbt debug — All checks passed
- Capture 12** — dbt run — Completed successfully (2 modèles créés)
- Capture 13** — Commit Git — Ajout des modèles dbt
- Capture 14** — Fichier schema.yml — définition des tests not_null
- Capture 15** — dbt test — PASS=3, Completed successfully
- Capture 16** — Commit Git — Ajout des tests simples dbt
- Capture 17** — Lancement de Dagster — dagster dev -f pipeline/orchestrate.py
- Capture 18** — Interface Dagster — Vue Overview
- Capture 19** — Interface Dagster — Liste des Jobs
- Capture 20** — Interface Dagster — Job ventes_pipeline avec les 4 ops
- Capture 21** — Interface Dagster — Job ventes_pipeline avec les 4 ops
- Capture 22** — Fichier ci.yml — configuration GitHub Actions
- Capture 23** — GitHub Actions — CI Pipeline déclenchée
- Capture 24** — GitHub Actions — CI réussie (workflow vert)
- Capture 25** — Commit Git — Ajout d'une CI simple
- Capture 26** — git log --oneline — Historique complet des commits
- Capture 27** — git push — Envoi du projet vers GitHub
- Capture 28** — Dépôt GitHub tp_chapitre2_pipeline — Référentiel en ligne
- Figure 1** — Schéma coloré du pipeline MLOps — CSV → Ingestion → DuckDB → Validation → dbt → Dagster → CI

1. Introduction et objectif du TP

Ce TP a pour objectif de transformer un traitement de données simple en un pipeline structuré et industrialisable. Plutôt que d'utiliser un simple notebook ou un script unique, nous construisons une architecture composée de plusieurs étapes séparées, versionnées et orchestrées.

Le pipeline suit la logique suivante : CSV → Ingestion → Validation → Transformation → Tests qualité → Orchestration

2. Outils utilisés

Outil	Rôle principal dans le TP
Python	Langage d'implémentation des scripts
venv	Environnement virtuel reproductible
pip	Installation des dépendances
requirements.txt	Sauvegarde et reproductibilité des dépendances
Git	Versionnement du projet
CSV (ventes.csv)	Source brute de données — entrée initiale
pandas	Lecture et manipulation des données
DuckDB	Moteur de stockage analytique local
dlt	Outil d'ingestion — 1ère étape du pipeline
dbt	Transformation SQL des données
dbt test	Contrôles qualité automatiques
Dagster	Orchestration du workflow d'exécution
GitHub Actions	Continuous Integration (CI)
.gitignore	Nettoyage du dépôt Git
YAML	Configuration des outils

3. Structure du projet

L'arborescence du projet créé est la suivante :

```
tp_chapitre2_pipeline/
├── data/
│   └── ventes.csv
├── pipeline/
│   ├── ingest.py
│   ├── validate.py
│   └── orchestrate.py
├── dbt_pipeline/
│   ├── dbt_project.yml
│   ├── profiles.yml
│   └── models/
│       ├── ventes_clean.sql
│       ├── ventes_resume.sql
│       └── schema.yml
├── .github/
│   └── workflows/
│       └── ci.yml
├── requirements.txt
└── .gitignore
```

4. Étapes réalisées

4.1 Préparation de l'environnement

Création du dossier du projet et de l'environnement virtuel :

```
mkdir tp_chapitre2_pipeline
cd tp_chapitre2_pipeline
python3 -m venv .venv
source .venv/bin/activate
pip install dlt dagster dagster-webserver dbt-core dbt-duckdb pandas duckdb
pyyaml
pip freeze > requirements.txt
```

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>pip freeze > requirements.txt

(.venv) C:\Users\PC\tp_chapitre2_pipeline>type requirements.txt
agate==1.9.1
alembic==1.18.4
annotated-types==0.7.0
antlr4-python3-runtime==4.13.2
anyio==4.13.0
attrs==26.1.0
babel==2.18.0
backoff==2.2.1
certifi==2026.5.20
charset-normalizer==3.4.7
click==8.4.1
colorama==0.4.6
coloredlogs==14.0
daff==1.4.2
dagster==1.13.7
dagster-graphql==1.13.7
dagster-pipes==1.13.7
dagster-shared==1.13.7
dagster-webserver==1.13.7
dbt-adapters==1.24.2
dbt-common==1.38.0
dbt-core==1.11.11
dbt-duckdb==1.10.1
dbt-extractor==0.6.0
dbt-protos==1.0.514
dbt-semantic-interfaces==0.9.0
```

Capture 1 : pip freeze > requirements.txt

4.2 Initialisation Git

```
git init
git config --global init.defaultBranch main
git add .
git commit -m "Initialisation du projet pipeline"
```

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>git init
Initialized empty Git repository in C:/Users/PC/tp_chapitre2_pipeline/.git/

(.venv) C:\Users\PC\tp_chapitre2_pipeline>git config --global init.defaultBranch main

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 2 : git init et configuration de la branche main

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>git commit -m "Initialisation du projet pipeline"
[master (root-commit) 3b5ff6c] Initialisation du projet pipeline
 2 files changed, 131 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 requirements.txt

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 3 : Premier commit Git — Initialisation du projet

4.3 Étape 1 — Ingestion (pipeline/ingest.py)

Ce script lit le fichier CSV avec pandas et charge les données brutes dans DuckDB :

```
import pandas as pd
import duckdb
from pathlib import Path

csv_path = Path("data/ventes.csv")
db_path = "ventes.duckdb"

df = pd.read_csv(csv_path)
con = duckdb.connect(db_path)
"CREATE OR REPLACE TABLE ventes_raw AS SELECT * FROM df"
con.close()
print("Ingestion terminée : ventes_raw créée dans DuckDB")
```

```
date,produit,categorie,quantite,prix_unitaire,ville
2026-01-01,Clavier,Informatique,2,300,Casablanca
2026-01-02,Souris,Informatique,5,120,Rabat
2026-01-03,Ecran,Informatique,1,2200,Casablanca
2026-01-04,Chaise,Mobilier,3,850,Marrakech
2026-01-05,Bureau,Mobilier,1,1500,Rabat
2026-01-06,Clavier,Informatique,4,300,Tanger
2026-01-07,Chaise,Mobilier,2,850,Casablanca|
```

Capture 4 : Contenu du fichier ventes.csv

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>python pipeline/ingest.py
Ingestion terminée : ventes_raw créée dans DuckDB

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 5 : Exécution de python pipeline/ingest.py — Ingestion terminée

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>git commit -m "Ajout d
e l'étape d'ingestion"
[master 88298b5] Ajout de l'étape d'ingestion
 1 file changed, 14 insertions(+)
 create mode 100644 pipeline/ingest.py

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 6 : Commit Git — Ajout de l'étape d'ingestion

4.4 Étape 2 — Validation minimale (pipeline/validate.py)

Ce script vérifie la présence des colonnes obligatoires et l'absence de valeurs nulles :

```
import duckdb
required_columns =
{"date", "produit", "categorie", "quantite", "prix_unitaire", "ville"}
con = duckdb.connect("ventes.duckdb")
columns = con.execute("DESCRIBE ventes_raw").fetchall()
existing_columns = {col[0] for col in columns}
missing = required_columns - existing_columns
if missing: raise ValueError(f"Colonnes manquantes : {missing}")
# Vérification des valeurs nulles...
print("Validation réussie : schéma et qualité minimale OK")
```

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>python pipeline/valida
te.py
Validation réussie : schéma et qualité minimale OK

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 7 : Exécution de python pipeline/validate.py — Validation réussie

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>git commit -m "Ajout d
e l'étape de validation minimale"
[master b62e512] Ajout de l'étape de validation minimale
 1 file changed, 23 insertions(+)
 create mode 100644 pipeline/validate.py

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 8 : Commit Git — Ajout de l'étape de validation minimale

4.5 Étape 3 — Transformation avec dbt

Modèle ventes_clean.sql — nettoyage et calcul du chiffre d'affaires :

```
select date, produit, categorie, quantite, prix_unitaire, ville,
       quantite * prix_unitaire as chiffre_affaires
from ventes_raw
where quantite > 0 and prix_unitaire > 0
```

Modèle ventes_resume.sql — agrégation par catégorie :

```
select categorie, sum(quantite) as total_quantite, sum(chiffre_affaires) as
total_ca
from {{ ref('ventes_clean') }}
```



```
group by categorie
```

```
select
    date,
    produit,
    categorie,
    quantite,
    prix_unitaire,
    ville,
    quantite * prix_unitaire as chiffre_affaires
from ventes_raw
where quantite > 0 and prix_unitaire > 0
```

Capture 9 : Modèle ventes_clean.sql

```
select
    categorie,
    sum(quantite) as total_quantite,
    sum(chiffre_affaires) as total_ca
from {{ ref('ventes_clean') }}
group by categorie
```

Capture 10 : Modèle ventes_resume.sql

```
cd dbt_pipeline
dbt debug --profiles-dir .
dbt run --profiles-dir .
```

```
11:55:18 config_options: None
11:55:18 extensions: None
11:55:18 settings: None
11:55:18 external_root: .
11:55:18 use_credential_provider: None
11:55:18 attach: None
11:55:18 filesystems: None
11:55:18 remote: None
11:55:18 plugins: None
11:55:18 disable_transactions: False
11:55:18 Registered adapter: duckdb=1.8.0
11:55:25 Connection test: [OK connection ok]

11:55:25 All checks passed!

(.venv) C:\Users\PC\tp_chapitre2_pipeline\dbt_pipeline>
```

Capture 11 : dbt debug — All checks passed


```
(.venv) C:\Users\PC\tp_chapitre2_pipeline\dbt_pipeline>dbt run -
-profiles-dir .
11:56:02 Running with dbt=1.8.0
11:56:02 Registered adapter: duckdb=1.8.0
11:56:03 Unable to do partial parsing because saved manifest no
t found. Starting full parse.
11:56:06 Found 2 models, 421 macros
11:56:06
11:56:06 Concurrency: 1 threads (target='dev')
11:56:06
11:56:06 1 of 2 START sql table model main.ventes_clean .....
..... [RUN]
11:56:06 1 of 2 OK created sql table model main.ventes_clean ..
..... [OK in 0.12s]
11:56:06 2 of 2 START sql table model main.ventes_resume .....
..... [RUN]
11:56:06 2 of 2 OK created sql table model main.ventes_resume .
..... [OK in 0.07s]
11:56:06
11:56:06 Finished running 2 table models in 0 hours 0 minutes a
nd 0.45 seconds (0.45s).
11:56:06
11:56:06 Completed successfully
11:56:06
11:56:06 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2

(.venv) C:\Users\PC\tp_chapitre2_pipeline\dbt_pipeline>
```

Capture 12 : dbt run — Completed successfully (2 modèles créés)

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>git commit -m "Ajout d
es modèles dbt"
[master 310ed75] Ajout des modèles dbt
6 files changed, 310 insertions(+)
create mode 100644 dbt_pipeline/.user.yml
create mode 100644 dbt_pipeline/dbt_project.yml
create mode 100644 dbt_pipeline/logs/dbt.log
create mode 100644 dbt_pipeline/models/ventes_clean.sql
create mode 100644 dbt_pipeline/models/ventes_resume.sql
create mode 100644 dbt_pipeline/profiles.yml

(.venv) C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 13 : Commit Git — Ajout des modèles dbt

```
cd ..
```

4.6 Étape 4 — Tests qualité avec dbt test

Définition des tests dans schema.yml (not_null sur les colonnes critiques), puis exécution :

```

version: 2

models:
  - name: ventes_clean
    columns:
      - name: produit
        tests:
          - not_null
      - name: quantite
        tests:
          - not_null

  - name: ventes_resume
    columns:
      - name: categorie
        tests:
          - not_null

```

Capture 14 : Fichier schema.yml — définition des tests not_null

```

cd dbt_pipeline
dbt test --profiles-dir .
cd ..

```

```

..... [PASS in 0.05s]
11:59:33
11:59:33 Finished running 3 data tests in 0 hours 0 minutes and
0.37 seconds (0.37s).
11:59:33
11:59:33 Completed successfully
11:59:33
11:59:33 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 TOTAL=3

(.venv) C:\Users\PC\tp_chapitre2_pipeline\dbt_pipeline>

```

Capture 15 : dbt test — PASS=3, Completed successfully

```

(.venv) C:\Users\PC\tp_chapitre2_pipeline>git commit -m "Ajout d
es tests simples dbt"
[master 609c0b1] Ajout des tests simples dbt
1 file changed, 17 insertions(+)
create mode 100644 dbt_pipeline/models/schema.yml

(.venv) C:\Users\PC\tp_chapitre2_pipeline>

```

Capture 16 : Commit Git — Ajout des tests simples dbt

4.7 Étape 5 — Orchestration avec Dagster

Le fichier `orchestrate.py` définit le pipeline complet comme un job Dagster :

```
from dagster import job, op
import os

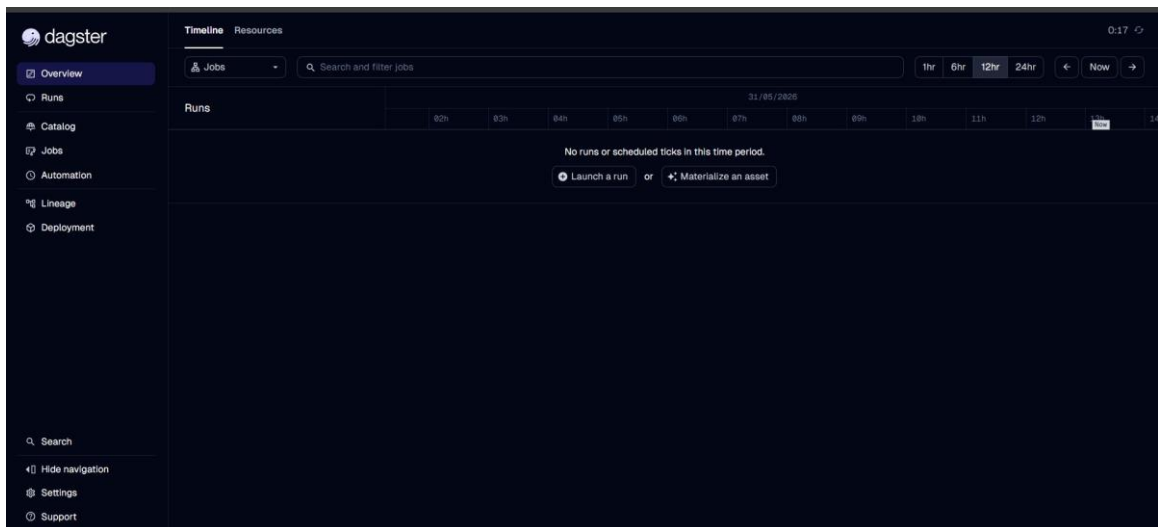
@op
def ingest(): os.system('python pipeline/ingest.py')
@op
def validate(): os.system('python pipeline/validate.py')
@op
def transform(): os.system('cd dbt_pipeline && dbt run --profiles-dir .')
@op
def test_data(): os.system('cd dbt_pipeline && dbt test --profiles-dir .')
@job
def ventes_pipeline():
    test_data(transform(validate(ingest())))
dagster dev -f pipeline/orchestrate.py
```

```
(.venv) C:\Users\PC\tp_chapitre2_pipeline>dagster dev -f pipeline/orchestrate.py
C:\Users\PC\tp_chapitre2_pipeline\.venv\Lib\site-packages\click\core.py:877: SupersessionWarning: Function 'dev_command' is superseded and its usage is discouraged. Use 'dg dev' instead.
  return callback(*args, **kwargs)
2026-05-31 13:15:26 +0100 - dagster - INFO - Using temporary directory C:\Users\PC\tp_chapitre2_pipeline\.tmp_dagster_home_23h9x9ws for storage. This will be removed when dagster dev exits.
2026-05-31 13:15:26 +0100 - dagster - INFO - To persist information across sessions, set the environment variable DAGSTER_HOME to a directory to use.
2026-05-31 13:15:28 +0100 - dagster - INFO - Launching Dagster services...
2026-05-31 13:15:34 +0100 - dagster.daemon - INFO - Instance is configured with the following daemons: ['AssetDaemon', 'BackfillDaemon', 'FreshnessDaemon', 'QueuedRunCoordinatorDaemon', 'SchedulerDaemon', 'SensorDaemon']
2026-05-31 13:15:35 +0100 - dagster - WARNING - C:\Users\PC\tp_chapitre2_pipeline\.venv\Lib\site-packages\dagster\_core\execution\compute_logs.py:53: UserWarning: WARNING: Compute log capture is disabled for the current environment. Set the environment variable 'PYTHONLEGACYWINDOWSTDIO' to enable.

  warnings.warn(WIN_PY36_COMPUTE_LOG_DISABLED_MSG)

2026-05-31 13:15:36 +0100 - dagster-webserver - INFO - Serving dagster-webserver on http://127.0.0.1:3000 in process 25724
```

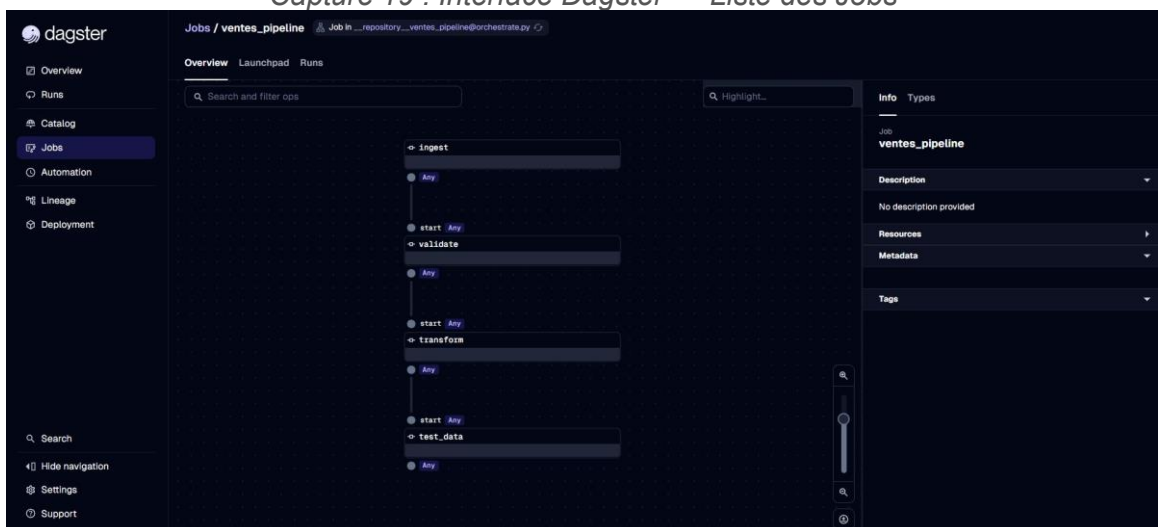
Capture 17 : Lancement de Dagster — `dagster dev -f pipeline/orchestrate.py`



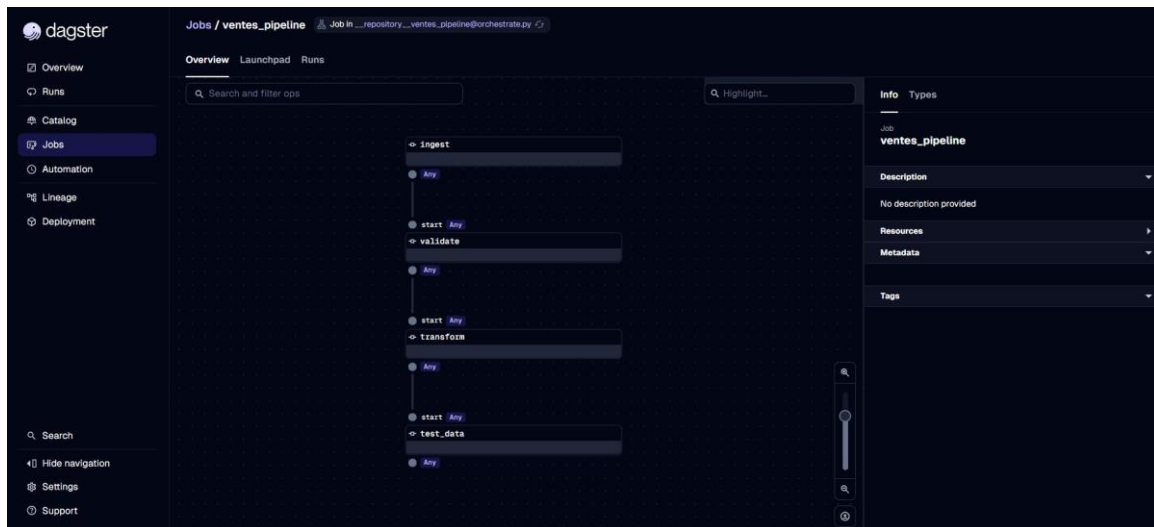
Capture 18 : Interface Dagster — Vue Overview



Capture 19 : Interface Dagster — Liste des Jobs



Capture 20 : Interface Dagster — Job ventes_pipeline avec les 4 ops



Capture 21 : Interface Dagster — Job `ventas_pipeline` avec les 4 ops

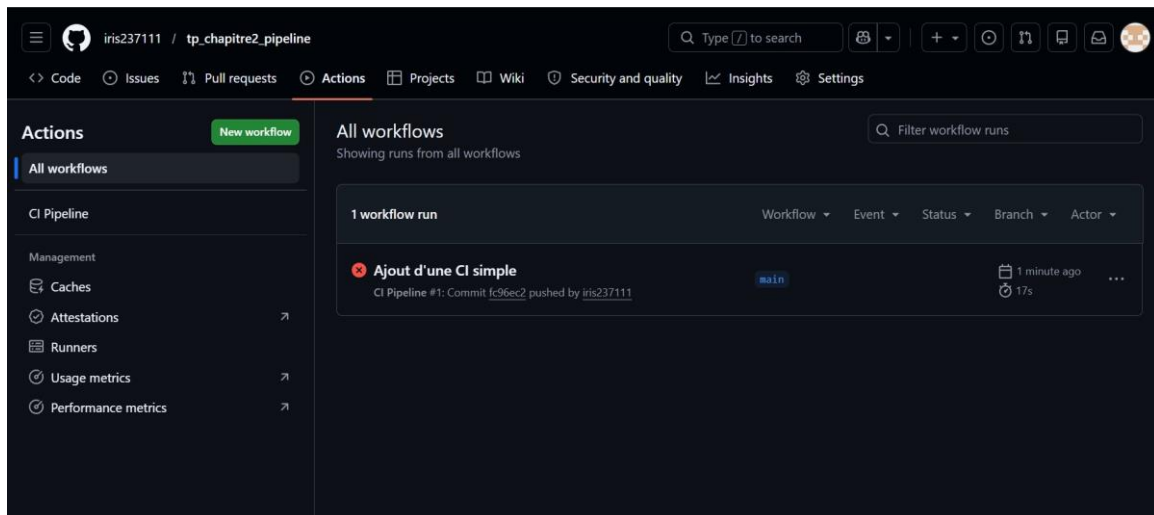
4.8 Étape 6 — CI avec GitHub Actions

Le fichier `.github/workflows/ci.yml` automatise les vérifications à chaque push sur main :

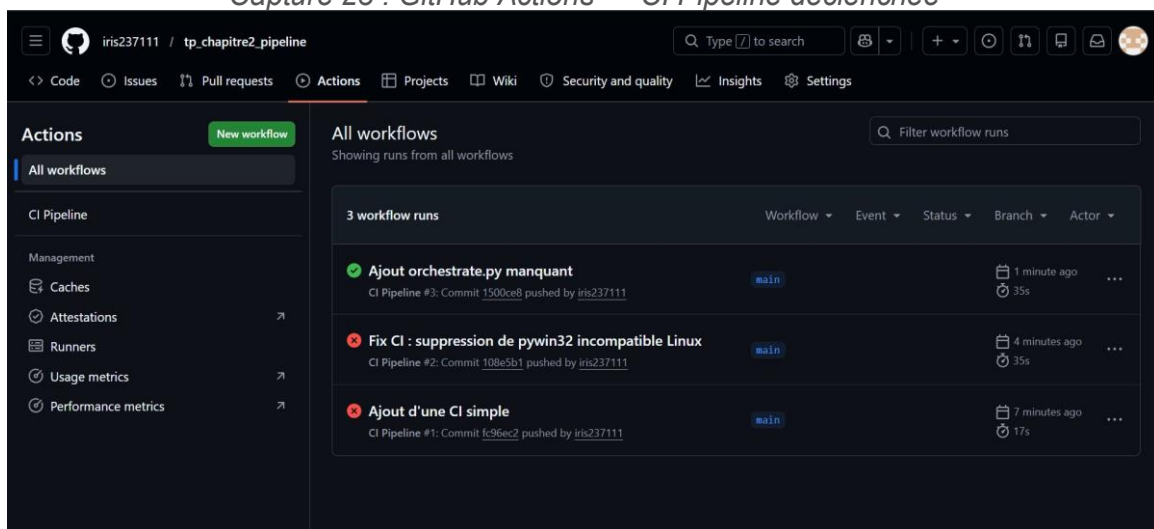
```
name: CI Pipeline
on:
  push:
    branches: [ main ]
jobs:
  validate-project:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: python -m py_compile pipeline/ingest.py pipeline/validate.py ...
```

```
name: CI Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  validate-project:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Syntax check
        run: |
          python -m py_compile pipeline/ingest.py pipeline/validate.py pipeline/orchestrate.py
```

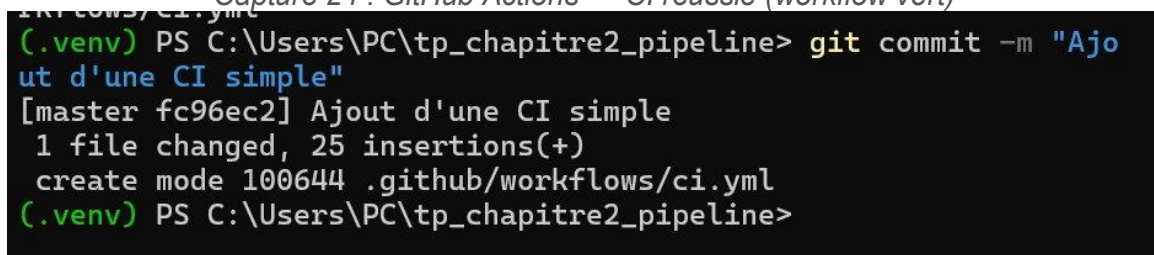
Capture 22 : Fichier `ci.yml` — configuration GitHub Actions



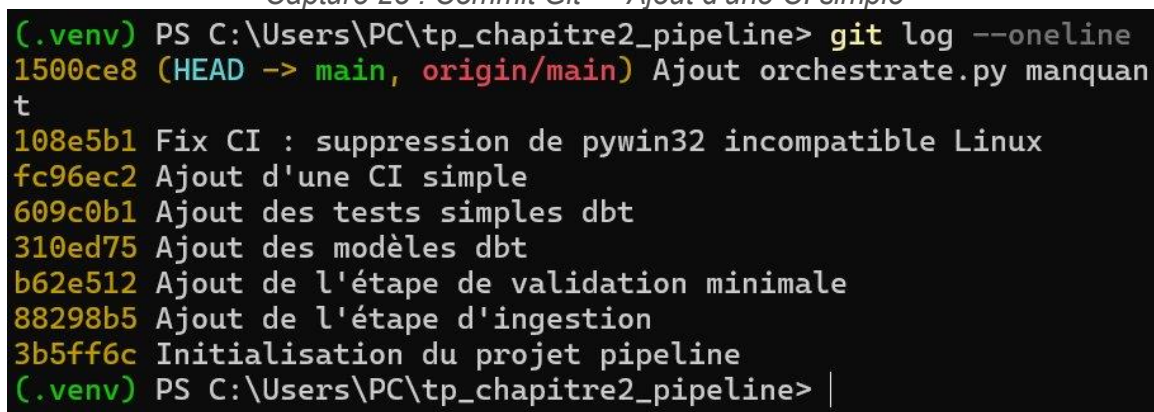
Capture 23 : GitHub Actions — CI Pipeline déclenchée



Capture 24 : GitHub Actions — CI réussie (workflow vert)



Capture 25 : Commit Git — Ajout d'une CI simple



Capture 26 : git log --oneline — Historique complet des commits

```
(.venv) PS C:\Users\PC\tp_chapitre2_pipeline> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 322 bytes | 322.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/iris237111/tp_chapitre2_pipeline.git
   fc96ec2..108e5b1  main -> main
(.venv) PS C:\Users\PC\tp_chapitre2_pipeline>
```

Capture 27 : git push — Envoi du projet vers GitHub

The screenshot shows the GitHub web interface for the repository `tp_chapitre2_pipeline` under the user `iris237111`. The repository is public. The page includes a navigation bar with links to Code, Issues, Pull requests, Actions, Projects, Wiki, Security and quality, Insights, and Settings. Below the navigation bar, there are buttons for Pin, Watch (0), Fork (0), and Star (0). The main content area features two cards: "Set up GitHub Copilot" and "Add collaborators to this repository". Below these cards, there is a "Quick setup" section with instructions on how to clone the repository using Desktop, HTTPS, or SSH. The SSH URL is `https://github.com/iris237111/tp_chapitre2_pipeline.git`. At the bottom, there is a section for creating a new repository on the command line, showing the command `echo "# tp_chapitre2_pipeline" >> README.md`.

Capture 28 : Dépôt GitHub `tp_chapitre2_pipeline` — Référentiel en ligne

5. Schéma du pipeline

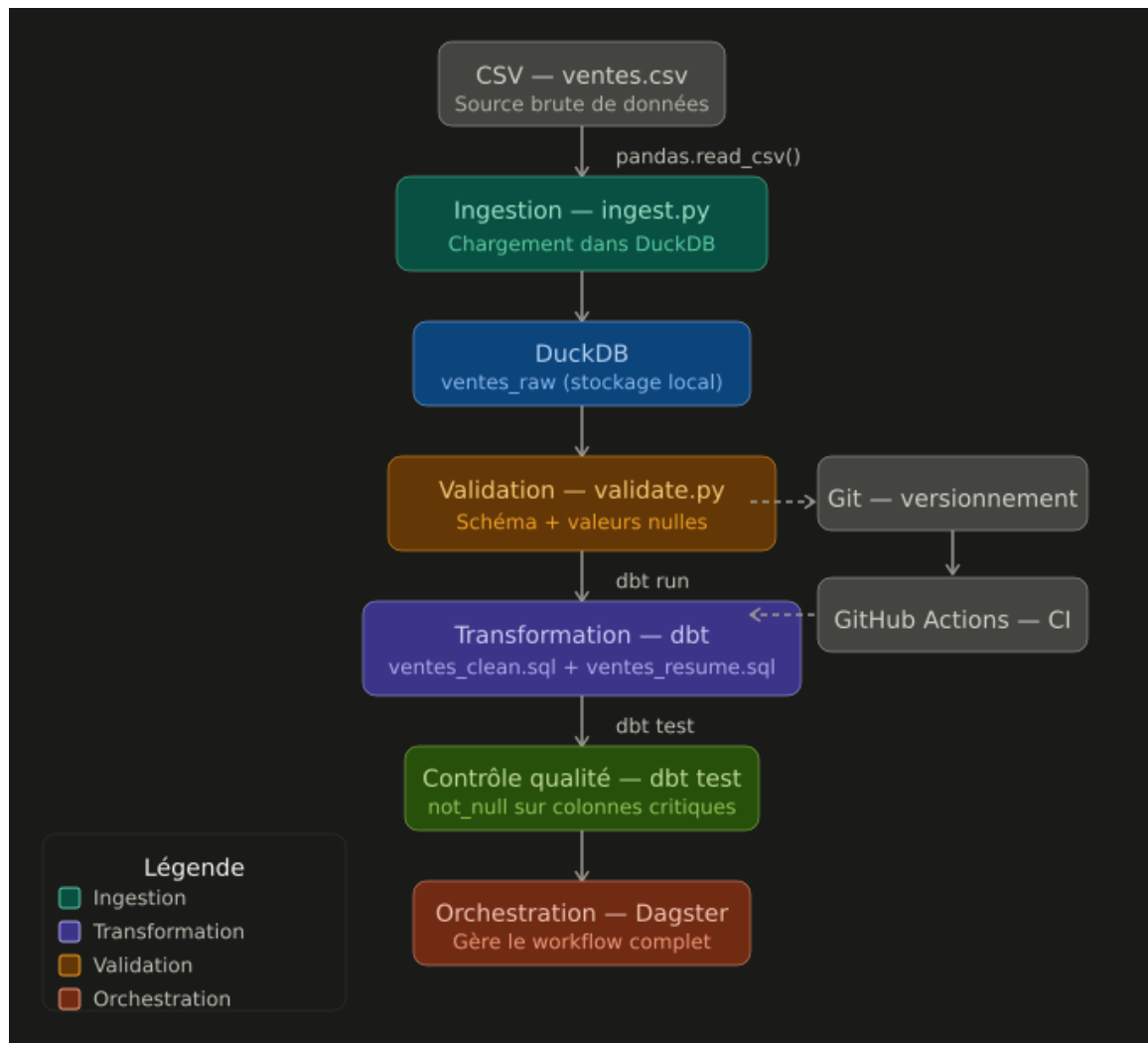


Figure 1 : Schéma coloré du pipeline MLOps — CSV → Ingestion → DuckDB → Validation → dbt → Dagster → CI

6. Réponses aux questions

Q1 : En quoi ce pipeline est-il plus structuré qu'un notebook unique ?

Un notebook unique mélange toutes les étapes dans un seul fichier sans séparation claire. Ce pipeline est plus structuré car :

- Chaque étape a son propre fichier et sa propre responsabilité (séparation des concerns)
- Les étapes sont indépendantes et peuvent être testées ou relancées séparément
- Le code est versionné avec Git, ce qui permet de tracer chaque modification
- L'orchestration avec Dagster garantit un ordre d'exécution reproductible
- Les résultats intermédiaires sont persistés dans DuckDB, pas seulement en mémoire

Q2 : Quel est le rôle de Git dans ce projet ?

Git est le système de versionnement du projet. Il permet de :

- Enregistrer chaque modification du code avec un message de commit explicatif
- Garder un historique complet de l'évolution du pipeline et revenir en arrière si besoin
- Travailler en équipe sans écraser le travail des autres
- Déclencher automatiquement la CI via GitHub Actions à chaque push

Un pipeline industrialisé doit être traçable et évolutif. Git est indispensable pour garantir ces propriétés.

Q3 : Quel est le rôle de requirements.txt ?

Le fichier requirements.txt liste toutes les bibliothèques Python et leurs versions exactes. Il garantit la reproductibilité technique : n'importe qui peut recréer le même environnement sur une autre machine avec la commande `pip install -r requirements.txt`. Sans ce fichier, le pipeline pourrait fonctionner sur une machine mais pas sur une autre à cause de versions incompatibles.

Q4 : Pourquoi ajouter une étape validate.py ?

L'étape de validation sert à détecter les problèmes de qualité le plus tôt possible dans le pipeline (principe du fail fast). Elle vérifie :

- La présence de toutes les colonnes obligatoires dans la table chargée
- L'absence de valeurs nulles dans les colonnes critiques (produit, quantité, prix_unitaire)

Si une donnée invalide passe sans être détectée, elle peut provoquer des erreurs silencieuses en aval. La validation protège l'intégrité du reste du pipeline.

Q5 : Quel est le rôle de dbt test ?

La commande dbt test exécute des tests automatiques définis dans schema.yml sur les modèles transformés. Elle vérifie par exemple que les colonnes critiques ne contiennent pas de valeurs nulles (test not_null). Contrairement à validate.py qui contrôle les données brutes, dbt test contrôle la qualité après transformation, assurant que les modèles produits sont fiables.

Q6 : Qu'apporte Dagster par rapport à une exécution manuelle ?

Sans Dagster, il faudrait lancer manuellement chaque commande dans le bon ordre.

Dagster apporte :

- La définition formelle de l'ordre des étapes via les dépendances entre ops
- Une interface web pour visualiser et lancer le pipeline graphiquement
- La gestion des échecs : si une étape échoue, les suivantes ne s'exécutent pas
- La traçabilité : chaque exécution est loguée avec son statut et ses sorties
- La possibilité de planifier les exécutions automatiquement

Q7 : Qu'apporte la CI ?

La CI via GitHub Actions apporte une couche d'automatisation et de sécurité :

- Vérification automatique de la syntaxe Python à chaque push sur la branche main
- Installation et vérification des dépendances dans un environnement propre
- Détection rapide des erreurs introduites par une modification de code
- Base pour étendre vers un CD (Continuous Deployment) complet

Q8 : Que manque-t-il encore pour un pipeline pleinement industrialisé ?

Ce pipeline est une bonne base, mais il lui manque :

- Une source de données réelle et dynamique (API, base de données de production)
- Un environnement de stockage cloud (S3, BigQuery, Snowflake) au lieu de DuckDB local
- Un système de monitoring et d'alertes en cas d'échec
- Un système de logging structuré et centralisé
- Des tests plus complets (unicité, plages de valeurs, références)
- Une gestion des secrets et credentials sécurisée
- Un déploiement en conteneur Docker pour une portabilité maximale
- Une planification automatique des exécutions (cron, Dagster schedules)

7. Conclusion

Ce TP nous a permis de comprendre la différence fondamentale entre un traitement de données artisanal et un pipeline structuré. En utilisant dlt/pandas pour l'ingestion, DuckDB pour le stockage local, dbt pour la transformation et les tests qualité, Dagster pour l'orchestration, et Git + GitHub Actions pour le versionnement et la CI, nous avons construit un pipeline qui respecte les grands principes du DataOps :

- Reproductibilité : grâce au venv et au requirements.txt
- Traçabilité : grâce à Git
- Qualité des données : grâce à validate.py et dbt test
- Orchestration : grâce à Dagster
- Automatisation : grâce à la CI GitHub Actions

Ce pipeline représente une première étape solide vers un système pleinement industrialisé, qui nécessiterait en production une infrastructure cloud, un monitoring avancé et une gestion des secrets.